Universidad Autónoma de Campeche

Reporte de programa
Práctica Machine Learning

Implementación de redes neuronales para clasificación, regresión y clasificación de series temporales.

Campo del formato	Información
Nombre de la asignatura	Pendiente de proporcionar
Nombre del profesor	Pendiente de proporcionar
Nombre del alumno	Ulises Zarate Concha
Matrícula	66575
Tema	Redes neuronales supervisadas y series de tiempo
Fecha de elaboración	15 de octubre de 2026
Lenguaje	Python

Reporte organizado conforme al archivo FORMATO REPORTE DE PROGRAMA (1).docx. Los dos campos pendientes deben completarse antes de entregar; no se asumieron datos de otra materia.

Código generado con asistencia de ChatGPT de OpenAI. Se documentan el prompt, los datasets, el entrenamiento y las pruebas realizadas.

Resumen

Se desarrollaron tres programas en Python para resolver problemas con datos públicos: reconocer el tipo de vino mediante su composición química, estimar la carga de calefacción de edificios y detectar defectos en obleas de silicio a partir de señales de sensores. Los dos primeros programas usan perceptrones multicapa; el tercero utiliza una red recurrente Echo State con una capa de lectura entrenable.

Los modelos se ajustaron sobre conjuntos de entrenamiento y validación. Después de seleccionar los parámetros se entrenaron otra vez con ambos conjuntos y se evaluaron sobre datos de prueba reservados. Los resultados medidos superan referencias sencillas: elegir siempre la clase mayoritaria o predecir siempre la media de la variable objetivo.

Programa	Resultado final en prueba	Referencia sencilla
Clasificación Wine	Accuracy 94.44 %; F1 macro 0.9407	Accuracy 38.89 %
Regresión Energy	R^2 0.9980; MAE 0.3378; RMSE 0.4509	MAE de la media 9.2721
Series Wafer	Accuracy 98.28 %; F1 defecto 0.9171	Accuracy normal siempre 89.21 %

Funcionalidades principales

Cada ejemplo permite cargar una muestra del conjunto de prueba, modificar sus valores, ejecutar la predicción y calcular métricas sobre todo el test. Los programas locales permiten repetir el ajuste, guardar pesos, generar gráficas y registrar las métricas. También se incluyen notebooks importables en Google Colab.

Qué se considera un resultado aceptable

Como criterio académico de revisión se emplea al menos 90 % de accuracy para Wine; R^2 mayor que 0.90 y un MAE inferior al predictor de la media para Energy; y F1 y recall de la clase defectuosa mayores que 0.85 para Wafer. Los tres resultados cumplen estos criterios. Son referencias de esta práctica, no umbrales industriales acordados con una empresa ni criterios usados para seleccionar repetidamente por el test.

La aplicación pública ejecuta la inferencia en Python con NumPy mediante Pyodide. El entrenamiento completo está en los archivos Python y los notebooks. Las tarjetas del sitio muestran resultados registrados; los botones de evaluación los calculan de nuevo con los pesos exportados.

Marco teórico

Aprendizaje supervisado y redes neuronales

En el aprendizaje supervisado se dispone de ejemplos con entradas y respuestas conocidas. La red aprende una relación que después se aplica a muestras nuevas. En clasificación la respuesta pertenece a una categoría, mientras que en regresión es un valor numérico continuo. El interés no es memorizar las respuestas del entrenamiento, sino mantener un buen desempeño sobre datos que no se utilizaron para ajustar el modelo [4].

Una neurona combina sus entradas mediante pesos y un sesgo. La suma pasa por una función de activación. Al conectar capas se forman transformaciones no lineales capaces de representar relaciones entre varias características. En un perceptrón multicapa, cada capa oculta recibe las salidas de la capa anterior. En esta práctica las capas ocultas usan ReLU, definida como el máximo entre cero y la entrada.

```
z = W x + b
ReLU(z) = max(0, z)
x_normalizada = (x - media_entrenamiento) / desviacion_entrenamiento
```

Pattern recognition y clasificación

El reconocimiento de patrones busca asociar una observación con una clase. Para Wine las observaciones son análisis químicos, con 13 variables y tres categorías de cultivar. El modelo termina en una salida softmax que transforma sus puntuaciones en probabilidades cuya suma es uno. La clase elegida corresponde a la probabilidad mayor. La pérdida de entropía cruzada penaliza las probabilidades bajas asignadas a la clase verdadera [1, 4].

La exactitud o accuracy es la proporción de observaciones clasificadas correctamente. Una matriz de confusión indica qué clases se mezclan. El F1 combina precisión y sensibilidad mediante su media armónica; el promedio macro da el mismo peso a cada clase. Por eso complementa a la accuracy cuando las clases tienen tamaños distintos.

Fitting y regresión

El ajuste de funciones aproxima una respuesta numérica a partir de varias entradas. En Energy se estima Y1, la carga de calefacción. La salida es lineal porque la predicción no es una categoría. Se normalizan tanto las entradas como el objetivo durante el aprendizaje, y la predicción se transforma de nuevo a la escala original antes de medir los errores. No se utiliza Y2 como variable de entrada [2, 4].

El MAE expresa el error absoluto medio en la escala de la variable objetivo. El RMSE penaliza más los errores grandes. R^2 compara el error cuadrático del modelo con la variación de los valores reales alrededor de su media; un valor cercano a uno indica un ajuste alto en el conjunto evaluado. Ese resultado por sí solo no garantiza desempeño fuera del dominio del dataset.

Marco teórico de series y evaluación

Series de tiempo y redes recurrentes

Una serie temporal conserva el orden de sus observaciones. En Wafer cada registro contiene 152 mediciones sucesivas de un sensor durante el procesamiento de una oblea. La tarea consiste en clasificar la secuencia completa como normal o defectuosa; no consiste en estimar el siguiente valor. Si se desordena la señal, cambia la información sobre la evolución del proceso [3].

Una red recurrente incorpora un estado interno que depende del estado anterior y de la entrada actual. Las Echo State Networks usan un reservorio recurrente con conexiones fijas, inicializadas al azar y controladas mediante un radio espectral. En esta implementación el estado se actualiza con tanh y un factor de fuga. Los pesos del reservorio no se entrenan: se aprende una capa de lectura MLP sobre resúmenes de sus estados. Es una variante de reservoir computing con lectura no lineal; no utiliza LSTM ni retropropagación a través del tiempo [5].

```
h_t = 0.5 h_(t-1) + 0.5 tanh(x_t W_in + h_(t-1) W + b)
caracteristicas = [h_final, media_temporal(h), maximo_temporal(h)]
24 estados x 3 resumenes = 72 entradas al clasificador MLP
```

La fuga 0.5 suaviza la actualización del estado; el radio espectral 0.8 controla la magnitud de la matriz recurrente. Estos valores describen la implementación y no son una prueba formal de la propiedad de estado de eco para cualquier entrada. La recurrencia sí procesa los pasos en orden, y la verificación confirma que invertir la señal cambia sus características.

Normalización y separación de los datos

Las variables tienen escalas diferentes; sin normalización, las magnitudes grandes pueden dominar el ajuste. StandardScaler calcula media y desviación con el conjunto usado para entrenar en cada etapa. Durante la selección, se ajusta únicamente sobre entrenamiento y se aplica sin cambios a validación. Tras elegir la configuración se reajusta con entrenamiento más validación. El test no participa en ese cálculo [6].

Se comparan tres configuraciones por problema con una semilla fija de 42. Wine selecciona por F1 macro, Energy por MAE y Wafer por F1 macro de validación. En un empate se conserva el primer candidato. El test se consulta para la evaluación final, después de decidir el candidato. No se emplea búsqueda por el desempeño del test.

Desbalance y límites de la validación

En Wafer la mayoría de secuencias es normal: predecir siempre normal ya obtiene 89.21 % de accuracy. Por eso se reportan sensibilidad y F1 de defectos, además de exactitud balanceada. Se conserva el train/test oficial del archivo; no hay marcas de tiempo para crear una partición cronológica de la fabricación. El orden temporal se refiere a las 152 lecturas dentro de cada muestra.

Equipo y software utilizado

El desarrollo y la evaluación se realizaron en el entorno Linux de esta sesión, mediante Python y ejecución en CPU. No se utilizó GPU. No se afirma que las pruebas se hayan ejecutado en el equipo personal del alumno; el modelo de CPU y la RAM del equipo del alumno no fueron proporcionados.

Componente	Versión o función
Python	3.12.14
NumPy	2.3.5; operaciones matriciales y red recurrente
scikit-learn	1.8.0; MLP, escaladores, separación y métricas
SciPy	1.17.0; optimización L-BFGS
Matplotlib	3.10.8; matrices y gráficas
Pyodide	0.27.7; Python WebAssembly con NumPy 2.0.2
Interfaz	HTML, CSS y JavaScript; Web Worker para los cálculos
Reporte	ReportLab; imágenes obtenidas de las salidas y gráficas reales

Contenido de la entrega

Carpeta o archivo	Contenido
programas	Entrenamiento, inferencia, tres lanzadores y tres notebooks
datos	Copias CSV de Wine, Energy y las dos particiones Wafer
resultados	Pesos JSON, métricas JSON, salida TXT y gráficas
evidencias	Registros visuales de salida y verificación Pyodide
requirements.txt	Dependencias con las versiones ejecutadas
README.md	Instalación, ejecución, estructura y límites
verificar.py	Equivalencia de inferencia y rechazo de entradas inválidas
ORIGEN_IA_Y_PROMPTS.txt	Atribución a IA y prompt utilizado

Los datos originales se localizaron en internet en UCI y en el archivo UCR/UEA. Las copias CSV permiten repetir el experimento sin descargar otra vez los datasets. Sus hashes SHA256 y las direcciones de descarga están en FUENTES_Y_DATOS.md. Los notebooks sí requieren internet para descargar el proyecto y las dependencias.

La prueba del intérprete Pyodide se ejecutó en Node con el mismo runtime WebAssembly y NumPy que usa el sitio. Las métricas coinciden con las de Python local. Esto verifica el motor numérico, pero no sustituye una revisión interactiva de todos los controles en un navegador.

Funcionalidad del programa de clasificación

Problema comercial: apoyar la identificación de un producto vinícola a partir de su composición. Wine contiene 178 análisis químicos de vinos de una región de Italia, correspondientes a tres cultivares. Las 13 entradas incluyen alcohol, ácido málico, cenizas, magnesio, fenoles, flavonoides e intensidad de color. Las etiquetas se codifican como 0, 1 y 2 y se presentan como Tipo 1, Tipo 2 y Tipo 3 [1].

El programa carga las muestras, crea una partición estratificada y ajusta un escalador junto con una red MLP. La partición es 106 ejemplos para entrenamiento inicial, 36 para validación y 36 para prueba. Tras la selección, el modelo final se entrena con los 142 ejemplos de desarrollo.

Capas ocultas	Alpha L2	F1 macro en validación
[16]	0.001	0.9740
[32, 16]	0.01	0.9740
[32]	0.1	1.0000

Configuración elegida: {'hidden': [32], 'alpha': 0.1, 'validation_f1_macro': 1.0}. ReLU en las capas ocultas, salida softmax, solver L-BFGS, máximo de 1500 iteraciones y semilla 42. L-BFGS no usa una tasa de aprendizaje de Adam; no se reporta una tasa que el optimizador no utiliza.

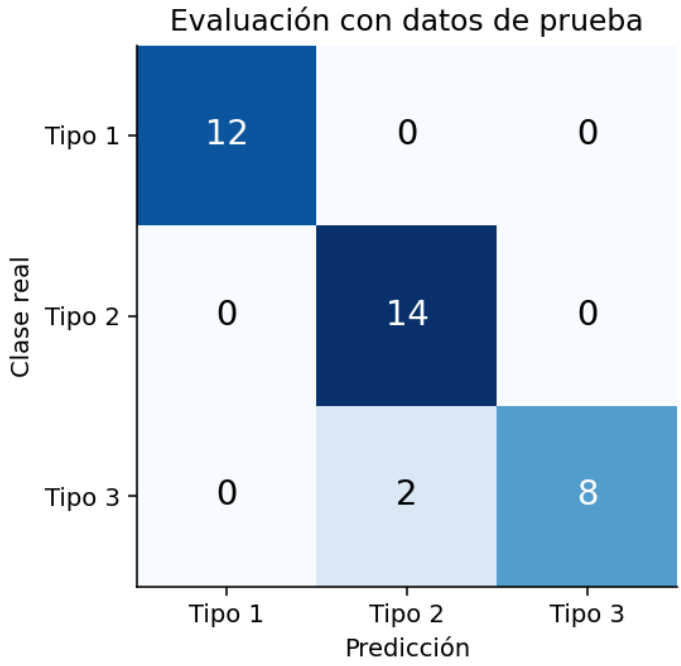


Figura 1. Matriz de confusión del test. Filas: clase real; columnas: predicción. Se acertaron 34 de 36 muestras; dos vinos de Tipo 3 se confundieron con Tipo 2. Accuracy 94.44 %, F1 macro 0.9407 y accuracy balanceada 93.33 %.

Funcionalidad del programa de regresión

Problema comercial: estimar la demanda de calefacción durante una comparación de diseños de edificios. Energy Efficiency contiene 768 configuraciones simuladas y ocho variables de entrada. No son 768 edificios medidos en campo. Se predice únicamente Y1; Y2, carga de enfriamiento, queda excluida para evitar que una respuesta relacionada facilite artificialmente el problema [2].

Entradas: compacidad relativa, superficie, área de muros, área de techo, altura total, orientación, área acristalada y distribución del acristalamiento. La partición aleatoria reserva 154 ejemplos para prueba. Se usan 460 para entrenamiento inicial y 154 para validación; el ajuste final emplea 614.

Capas ocultas	MAE validación	R^2 validación
[32, 16]	0.3429	0.9977
[64, 32]	0.3567	0.9972
[32]	0.2917	0.9982

Se eligió una capa oculta de 32 neuronas. Se utiliza ReLU, salida lineal, alpha L2 de 0.01, solver L-BFGS y un máximo de 2500 iteraciones. TransformedTargetRegressor normaliza el objetivo con estadísticas del entrenamiento y devuelve las predicciones a la escala original.

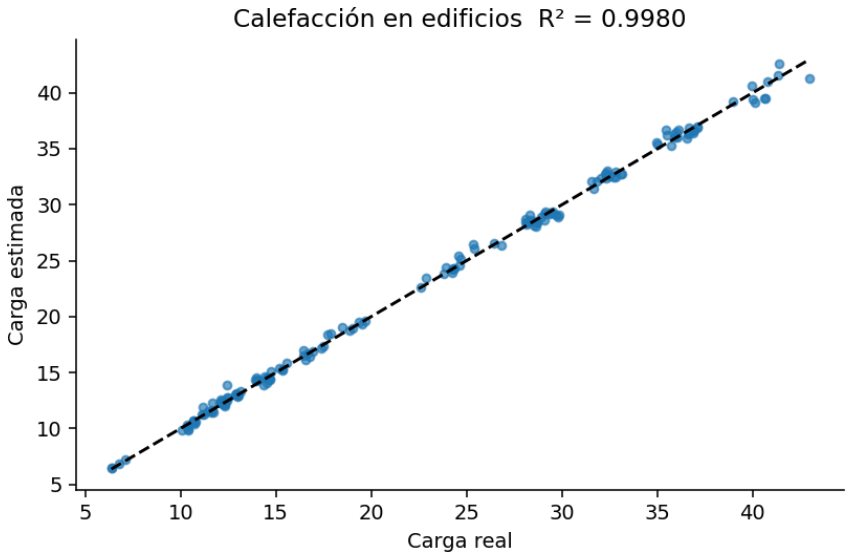


Figura 2. Predicciones y valores reales del test. La línea discontinua representa la coincidencia exacta. R^2 0.9980; MAE 0.3378; RMSE 0.4509. El MAE se informa en unidades de la variable Y1 del dataset, sin asumir una unidad física no especificada en la tabla de datos.

La partición puede compartir configuraciones geométricas similares entre desarrollo y test. El resultado demuestra ajuste dentro de este conjunto; una evaluación de edificios no vistos requeriría una partición por geometría y mediciones de campo.

Funcionalidad del programa de series

Problema industrial: identificar comportamientos anormales en un proceso de fabricación de semiconductores. Wafer reúne mediciones de sensores registradas durante el procesamiento de obleas de silicio. Se conserva la división original: 1000 secuencias de entrenamiento y 6164 de prueba. Cada secuencia tiene 152 observaciones [3].

La etiqueta original -1 se convierte a 1, Defectuosa; la etiqueta +1 se convierte a 0, Normal. El train original se divide de forma estratificada en 800 muestras para entrenamiento inicial y 200 para validación. El modelo final usa las 1000 muestras originales de entrenamiento.

Componente	Parámetro
Reservorio recurrente	24 unidades; tanh; fuga 0.5; radio espectral 0.8
Inicialización	Semilla 42; escala de entrada 0.8; sesgos entre -0.2 y 0.2
Características	Estado final, media y máximo temporal: 72 valores
Lectura entrenable	MLP con ReLU y salida sigmoide; alpha 0.1
Optimizador	L-BFGS; máximo de 1000 iteraciones

Capas ocultas de lectura	F1 macro en validación
[16]	1.0000
[32]	0.9695
[32, 16]	0.9695

Se seleccionó la lectura con una capa oculta de 16 neuronas. El reservorio permanece fijo durante los tres candidatos: se ajusta la lectura y no todos los parámetros recurrentes. Esta decisión reduce el costo de entrenamiento sin eliminar el procesamiento temporal.

La gráfica senales.png del ZIP muestra una secuencia real por clase. Conserva el orden de las 152 mediciones y la escala del sensor, sin atribuir una unidad física no documentada.

Resultados de la clasificación temporal

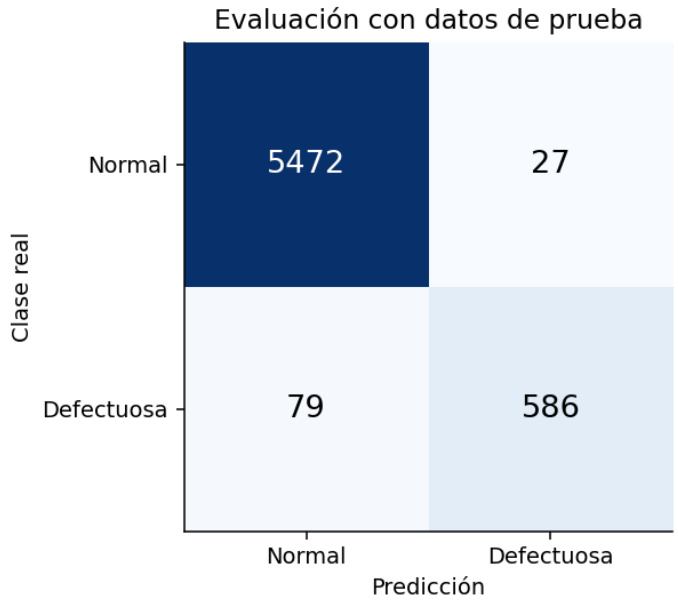


Figura 3. Matriz de confusión de Wafer sobre las 6164 secuencias de prueba. La clase positiva es Defectuosa.

Indicador	Resultado
Accuracy	98.2803 %
F1 macro	0.9537
Accuracy balanceada	93.8147 %
Recall de defectos	586 / 665 = 88.1203 %
Precisión de defectos	586 / (586 + 27) = 95.5954 %
F1 de defectos	0.9171
Falsos negativos	79 defectos se clasificaron como normales
Falsos positivos	27 piezas normales se clasificaron como defectuosas

De 665 secuencias defectuosas se detectaron 586. Quedaron 79 defectos sin detectar, por lo que la exactitud alta no debe interpretarse como eliminación del riesgo industrial. En un uso real habría que definir el costo de omitir un defecto y revisar el umbral de clasificación, validándolo con nuevos datos.

El sistema devuelve la clase y su probabilidad estimada. Esas probabilidades no fueron calibradas. Se utiliza el umbral estándar de 0.5 de la lectura binaria; no se optimizó el umbral consultando el test.

Instrucciones de uso

Ejecución desde los enlaces públicos

1. Abre el enlace del ejemplo. 2. Elige el número de una muestra y pulsa Cargar muestra. 3. Revisa sus valores y la respuesta real. 4. Pulsa Ejecutar predicción. 5. Para revisar el desempeño general, pulsa Evaluar todo el test. La primera ejecución descarga el intérprete Python y NumPy desde CDN; se necesita conexión a internet.

Programa 1 Clasificación

Programa 2 Regresión

Programa 3 Series de tiempo

Datos que debe ingresar el usuario

Ejemplo	Entrada	Salida esperada
Clasificación	13 valores químicos separados por comas	Tipo de vino y probabilidades de 3 clases
Regresión	8 valores del edificio, en el orden documentado	Carga estimada de calefacción en escala Y1
Series	152 valores del sensor en orden temporal	Normal o Defectuosa y sus probabilidades

Las entradas con cantidad incorrecta de números o con valores no finitos se rechazan con un mensaje. La validación comprueba formato, cantidad y finitud; no garantiza que un valor químico o geométrico sea físicamente plausible. Para probar valores propios deben conservarse la escala y el dominio del dataset.

Ejecución local y reentrenamiento

```
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
python -m pip install -r requirements.txt
python programas/entrenar.py todos
# O un caso: python programas/entrenar.py series
python verificar.py
```

Ejecuta los comandos desde la carpeta descomprimida. El programa crea o actualiza resultados con los pesos JSON, las métricas y las gráficas. No necesitas descargar otra vez los datos porque las copias CSV están incluidas.

Uso en Google Colab

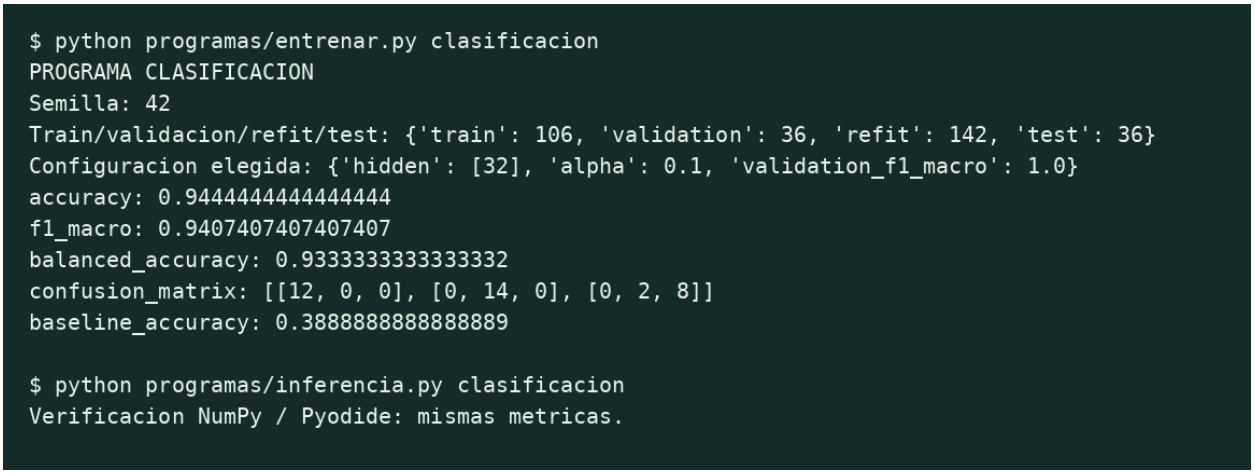
Abre colab.research.google.com, selecciona Archivo, Subir cuaderno y elige el notebook del ejemplo en programas. Después selecciona Entorno de ejecución, Ejecutar todas. El cuaderno descarga el ZIP público, instala dependencias y vuelve a entrenar el caso. No requiere acceso a una cuenta de Drive del autor ni claves de una API.

Ejemplo de uso de clasificación

Ejemplo reproducible: muestra 1 del test. Ingresa sus 13 variables químicas, en el orden usado por el modelo. El archivo clasificacion_prueba.csv contiene primero la etiqueta y después las entradas; no se ingresa la etiqueta al modelo.

```
14.1, 2.16, 2.3, 18, 105, 2.95,
3.32, 0.22, 2.38, 5.75, 1.25, 3.17, 1510
```

Clase real: Tipo 1. Predicción calculada: Tipo 1. Probabilidades: Tipo 1: 0.999997, Tipo 2: 0.000000, Tipo 3: 0.000003.



Registro visual de salida real. La imagen transcribe fielmente los valores guardados al ejecutar el programa; no es una fotografía de pantalla ni una captura de un IDE. La salida íntegra y las gráficas están incluidas en el ZIP para reproducir la evidencia.

Comprobación del ejemplo: el botón Evaluar todo el test debe devolver las métricas registradas en este reporte, salvo pequeñas diferencias de redondeo. Se verificó que la inferencia NumPy y el runtime Pyodide reproducen las métricas finales.

Ejemplo de uso de regresión

Ejemplo reproducible: muestra 1 del test. Se ingresan las ocho características del edificio; la carga de calefacción real se muestra solo como referencia de comprobación.

```
0.62, 808.5, 367.5, 220.5, 3.5, 2, 0.4, 3
```

Valor real Y1: 16.4700. Valor estimado: 16.5263. Error absoluto de esta muestra: 0.0563. El promedio de error de todo el test se calcula de forma independiente.

```
$ python programas/entrenar.py regresion
PROGRAMA REGRESION
Semilla: 42
Train/validacion/refit/test: {'train': 460, 'validation': 154, 'refit': 614, 'test':
154}
Configuracion elegida: {'hidden': [32], 'alpha': 0.01, 'validation_mae':
0.291741071741068, 'validation_r2': 0.9982246852075669}
r2: 0.9980494534695058
mae: 0.33783567172941464
rmse: 0.45089928397562384
baseline_mae: 9.272103050044418

$ python programas/inferencia.py regresion
Verificacion NumPy / Pyodide: mismas metricas.
```

Registro visual de salida real. La imagen transcribe fielmente los valores guardados al ejecutar el programa; no es una fotografía de pantalla ni una captura de un IDE. La salida íntegra y las gráficas están incluidas en el ZIP para reproducir la evidencia.

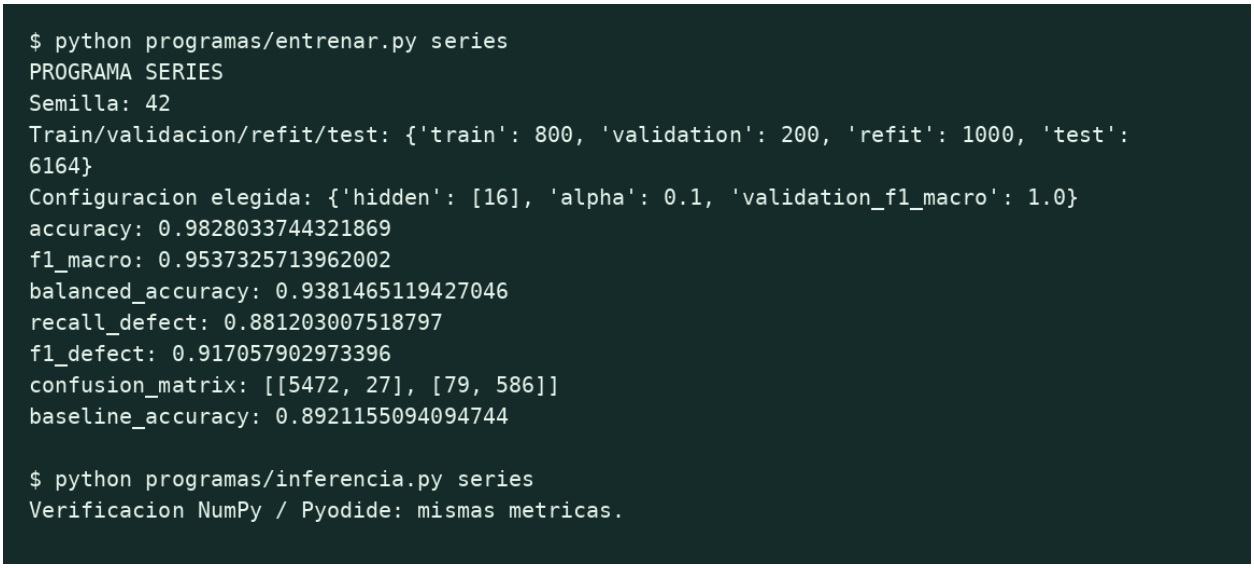
Comprobación del ejemplo: el botón Evaluar todo el test debe devolver las métricas registradas en este reporte, salvo pequeñas diferencias de redondeo. Se verificó que la inferencia NumPy y el runtime Pyodide reproducen las métricas finales.

Ejemplo de uso de series

Ejemplo reproducible: muestra 7 del test. En el sitio escribe ese número, pulsa Cargar muestra y Ejecutar predicción. Se cargan las 152 lecturas completas; estos son sus primeros 12 valores para identificar la secuencia:

```
-1.29030, -1.55046, -1.50562, -1.51139, -1.54913, -1.46204,
-1.63058, -1.24645, -0.75714, 0.14599, 1.40850, 1.72772, ...
```

Clase real: Defectuosa. Predicción calculada: Defectuosa. Probabilidad estimada de defecto: 0.999986. El clasificador recibe la secuencia completa y no solo el fragmento mostrado.



Registro visual de salida real. La imagen transcribe fielmente los valores guardados al ejecutar el programa; no es una fotografía de pantalla ni una captura de un IDE. La salida íntegra y las gráficas están incluidas en el ZIP para reproducir la evidencia.

Comprobación del ejemplo: el botón Evaluar todo el test debe devolver las métricas registradas en este reporte, salvo pequeñas diferencias de redondeo. Se verificó que la inferencia NumPy y el runtime Pyodide reproducen las métricas finales.

Código relevante de clasificación y regresión

El código completo está en programas/entrenar.py. Los fragmentos siguientes muestran los componentes centrales, con los mismos parámetros del programa. Los nombres abreviados representan variables ya cargadas en el archivo completo.

MLP para clasificación

```
m = make_pipeline(
    StandardScaler(),
    MLPClassifier(
        hidden_layer_sizes=h, alpha=alpha,
        solver="lbfgs", max_iter=1500, random_state=42
    )
)
m.fit(X[train], y[train])
score = f1_score(
    y[val], m.predict(X[val]), average="macro"
)
```

La búsqueda recorre tres pares de capas y alpha. Selecciona el F1 macro de validación más alto y después reajusta el pipeline con desarrollo. El escalador está dentro del pipeline, de modo que se ajusta con los mismos ejemplos utilizados para entrenar la red.

MLP para fitting con transformación del objetivo

```
m = TransformedTargetRegressor(
    regressor=make_pipeline(
        StandardScaler(),
        MLPRegressor(
            hidden_layer_sizes=(32,), alpha=0.01,
            solver="lbfgs", max_iter=2500, random_state=42
        )
    ),
    transformer=StandardScaler()
)
m.fit(X[dev], y[dev])
p = m.predict(X[test])
mae = mean_absolute_error(y[test], p)
r2 = r2_score(y[test], p)
```

La transformación del objetivo se aprende con desarrollo en el ajuste final. predict revierte la transformación automáticamente. Los errores se calculan sobre los valores originales, lo que permite compararlos con la media constante de desarrollo.

Exportación e inferencia

Los modelos se exportan como matrices de pesos y sesgos en JSON. La inferencia de programas/inferencia.py aplica esas matrices con NumPy, ReLU y la activación de salida. El sitio ejecuta ese archivo Python en un Web Worker de Pyodide. No usa respuestas precalculadas para los valores ingresados por el usuario.

Código relevante de la red recurrente

El reservorio de programas/inferencia.py se utiliza tanto al entrenar como al predecir. La semilla reconstruye las mismas conexiones y cada secuencia empieza con un estado cero; no se mezclan estados entre obleas.

```
def reservoir(X):
    X = np.asarray(X, dtype=float)
    n = 24
    rng = np.random.default_rng(42)
    wi = rng.normal(0, .8, (1, n))
    w = rng.normal(0, 1, (n, n))
    w *= .8 / np.max(np.abs(np.linalg.eigvals(w)))
    bias = rng.uniform(-.2, .2, n)
    h = np.zeros((len(X), n))
    s = np.zeros_like(h)
    mx = np.full_like(h, -np.inf)
    for t in range(X.shape[1]):
        h = .5*h + .5*np.tanh(X[:,t:t+1]@wi + h@w + bias)
        s += h
        mx = np.maximum(mx, h)
    return np.hstack([h, s/X.shape[1], mx])
```

La expresión h@w incorpora memoria del paso anterior. Se concatenan tres resúmenes, conservando información de la evolución y de los máximos de la señal. El código no sustituye la serie por un promedio calculado antes del procesamiento recurrente.

Lectura entrenable y pruebas de consistencia

```
m = make_pipeline(
    StandardScaler(),
    MLPClassifier(
        hidden_layer_sizes=(16,), alpha=.1,
        solver="lbfgs", max_iter=1000, random_state=42
    )
)
m.fit(reservoir(X_train), y_train)
pred = m.predict(reservoir(X_test))
```

verificar.py compara las métricas de la inferencia JSON con las guardadas durante el entrenamiento, con tolerancia de 1e-8. Además comprueba el rechazo de dimensiones incorrectas, NaN e infinito, y confirma que invertir el orden de una señal cambia los estados. Las tres implementaciones superaron estas verificaciones.

El runtime Pyodide 0.27.7 con NumPy 2.0.2 también se ejecutó y reprodujo accuracy, F1, recall, R^2, MAE y RMSE. El registro íntegro está en evidencias/pyodide_verificado.txt. No se realizó una prueba visual interactiva de todos los controles del sitio.

Origen del código y conclusiones

Asistencia de inteligencia artificial

El código de entrenamiento, la inferencia, los notebooks y la interfaz se generaron con ChatGPT de OpenAI el 15 de octubre de 2026, a partir de la consigna del alumno. Los programas se ejecutaron y sus resultados se registraron en esta sesión. No se presenta el código generado como programación manual del alumno.

Prompt utilizado

Se utilizaron los siguientes fragmentos literales de la consigna. La versión documentada está en ORIGEN_IA_Y_PROMPTS.txt.

Implementar en c++, python o java, 1 ejemplo de redes neuronales de tipo pattern recognition (clasificación) para un problema de clasificación real, comercial o industrial. (buscar los datasets en internet) con sus parámetros ajustados y con resultados aceptables.

Implementar en c++, python o java, 1 ejemplo de redes neuronales de tipo fitting (regresión) para un problema de regresión real, comercial o industrial. (buscar los datasets en internet) con sus parámetros ajustados y con resultados aceptables.

Implementar en c++, python o java, 1 ejemplo de redes neuronales de tipo series de tiempo para un problema de clasificación real o industrial. (buscar los datasets en internet) con sus parámetros ajustados y con resultados aceptables.

Se permite usar frameworks, generar los códigos con IA generativa (poner su origen y los prompts usados). Los enlaces deben permitir correr dicho programa y contener las instrucciones de su uso.

No se tomó una implementación de un repositorio externo. Se consultó la documentación de las bibliotecas y las fuentes de datos. Los prompts orientan la generación; la evidencia de funcionamiento es la ejecución y la evaluación registrada, no la respuesta textual de la IA.

Conclusiones

Las tres implementaciones resolvieron las tareas planteadas con resultados aceptables en sus datasets. La clasificación de vinos conserva dos errores en el test; la regresión obtiene un error bajo dentro de configuraciones simuladas; la red recurrente detecta 586 de 665 defectos. Mostrar los errores permite interpretar mejor el alcance de cada resultado.

Para ampliar el trabajo conviene validar Wine con más muestras de origen independiente, separar Energy por geometrías y contrastarlo con edificios reales, y ajustar la decisión de Wafer con costos de error definidos por un proceso industrial. Esas mejoras requieren nuevas pruebas y no se atribuyen al experimento actual.

Bibliografía

[1] Wine. UCI Machine Learning Repository. Dataset de análisis químicos.

https://archive.ics.uci.edu/dataset/109/wine

[2] Tsanas, A. y Xifara, A. Energy Efficiency. UCI Machine Learning Repository.

https://archive.ics.uci.edu/dataset/242/energy+efficiency

[3] Wafer. UCR/UEA Time Series Classification Archive. Descripción y particiones oficiales.

https://www.timeseriesclassification.com/description.php?Dataset=Wafer

[4] scikit-learn. Neural network models supervised. Fundamentos de MLPClassifier y MLPRegressor.

https://scikit-learn.org/stable/modules/neural_networks_supervised.html

[5] Jaeger, H. 2001. The echo state approach to analysing and training recurrent neural networks. GMD Report 148.

https://www.ai.rug.nl/minds/uploads/EchoStatesTechRep.pdf

[6] scikit-learn. StandardScaler. Normalización mediante estadísticas del entrenamiento.

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html

[7] Pyodide. Getting started. Versión 0.27.7.

https://pyodide.org/en/0.27.7/usage/quickstart.html

[8] scikit-learn. load_wine. Procedencia de la copia del dataset usada.

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_wine.html

Fecha de consulta de fuentes y descarga de los datos: 15 de octubre de 2026. Los documentos de bibliotecas son la referencia del marco teórico y de las API; no se les atribuyen los resultados experimentales de esta práctica.

Enlaces de los programas desarrollados

Clasificacion: https://practica-ml-ulises-66575.equal-knoll-6193.chatgpt.site/?ejemplo=clasificacion

Regresion: https://practica-ml-ulises-66575.equal-knoll-6193.chatgpt.site/?ejemplo=regresion

Series: https://practica-ml-ulises-66575.equal-knoll-6193.chatgpt.site/?ejemplo=series

Código completo: descarga ZIP desde los programas públicos. Entrenamiento: programas/entrenar.py. Inferencia: programas/inferencia.py. Instrucciones adicionales, fuentes directas y hashes de los datos en el ZIP.